

PRACTICE SESSION

21.09.2025

Implementation of Principal Component Analysis in Python

- **Step 1: Importing Required Libraries**

- We import the necessary library like [pandas](#), [numpy](#), [scikit learn](#), [seaborn](#) and [matplotlib](#).

- import numpy as np
- import pandas as pd
- from sklearn.preprocessing import StandardScaler
- from sklearn.decomposition import PCA
- from sklearn.model_selection import train_test_split
- from sklearn.linear_model import LogisticRegression
- from sklearn.metrics import confusion_matrix
- import matplotlib.pyplot as plt
- import seaborn as sns

• Step 2: Creating Sample Dataset

- We make a small dataset with features Height, Weight, Age and Gender.

- data = {
 - 'Height': [170, 165, 180, 175, 160, 172, 168, 177, 162, 158],
 - 'Weight': [65, 59, 75, 68, 55, 70, 62, 74, 58, 54],
 - 'Age': [30, 25, 35, 28, 22, 32, 27, 33, 24, 21],
 - 'Gender': [1, 0, 1, 1, 0, 1, 0, 1, 0, 0] # 1 = Male, 0 = Female
- }

Output:

- df = pd.DataFrame(data)
- print(df)

	Height	Weight	Age	Gender
0	170	65	30	1
1	165	59	25	0
2	180	75	35	1
3	175	68	28	1
4	160	55	22	0
5	172	70	32	1
6	168	62	27	0
7	177	74	33	1
8	162	58	24	0
9	158	54	21	0

- **Step 3: Standardizing the Data**

- Since the features have different scales Height vs Age we standardize the data. This makes all features have mean = 0 and standard deviation = 1 so that no feature dominates just because of its units.

- `X = df.drop('Gender', axis=1)`
- `y = df['Gender']`
- `scaler = StandardScaler()`
- `X_scaled = scaler.fit_transform(df)`

- **Step 4: Applying PCA algorithm**

- Reduce the data from 3 features to 2 new features called principal components. These components capture most of the original information but in fewer dimensions.
- Split the data into 70% training and 30% testing sets.
- Train a logistic regression model on the reduced training data and predict gender labels on the test set.

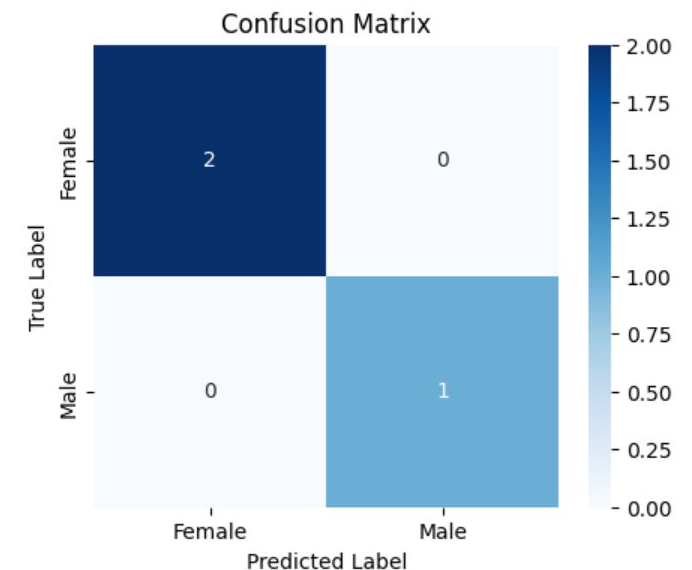
- `pca = PCA(n_components=2)`
- `X_pca = pca.fit_transform(X_scaled)`
- `X_train, X_test, y_train, y_test = train_test_split(X_pca, y, test_size=0.3, random_state=42)`
- `model = LogisticRegression()`
- `model.fit(X_train, y_train)`
- `y_pred = model.predict(X_test)`

• Step 5: Evaluating with Confusion Matrix

- The [confusion matrix](#) compares actual vs predicted labels. This makes it easy to see where predictions were correct or wrong.

- `cm = confusion_matrix(y_test, y_pred)`
- `plt.figure(figsize=(5,4))`
- `sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Female', 'Male'], yticklabels=['Female', 'Male'])`
- `plt.xlabel('Predicted Label')`
- `plt.ylabel('True Label')`
- `plt.title('Confusion Matrix')`
- `plt.show()`

Output:



- **Step 6: Visualizing PCA Result**

- `y_numeric = pd.factorize(y)[0]`
- `plt.figure(figsize=(12, 5))`
- `plt.subplot(1, 2, 1)`
- `plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=y_numeric, cmap='coolwarm', edgecolor='k', s=80)`
- `plt.xlabel('Original Feature 1')`
- `plt.ylabel('Original Feature 2')`
- `plt.title('Before PCA: Using First 2 Standardized Features')`
- `plt.colorbar(label='Target classes')`
- `plt.subplot(1, 2, 2)`
- `plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y_numeric, cmap='coolwarm', edgecolor='k', s=80)`
- `plt.xlabel('Principal Component 1')`
- `plt.ylabel('Principal Component 2')`
- `plt.title('After PCA: Projected onto 2 Principal Components')`
- `plt.colorbar(label='Target classes')`
- `plt.tight_layout()`
- `plt.show()`

- **Before PCA:** Shows the **original standardized data** plotted using the first two features. There is **no guarantee of clear separation** between classes as these are raw input dimensions.
- **After PCA:** Displays the **transformed data** using the **top 2 principal components**. These new components capture the **maximum variance** often showing better **class separation and structure** making it easier to analyze or model

Output:

